Université Versailles-Saint-Quentin

Paris-Saclay



Spécialité :
Calcul Haute Performance, Simulation

Module :

April 28, 2026

Rapport : TOP

Présenté par:
- Masten OUNNAS.
- Prince-Marcel MASSAMBA

Lien git:
https://github.com/TaskMaster99/TOP-26-M1-CHPS

Contents

1 Configuration du Système **3**
1 Matériel . . . 3
2 Logiciel . . . 3
3 Note sur la topologie NUMA . . . 3

2 Méthodologie **5**

3 Optimisation 1 : Suppression du verrou de synchronisation **6**
1 Identification du problème . . . 6
2 Correction et mesure . . . 6

4 Optimisation 2 : Restructuration mémoire AoS → SoA **7**
1 Diagnostic . . . 7
2 Implémentation . . . 7

5 Optimisation 3 : Réduction des calculs redondants **8**

6 Optimisation 4 : Suppression des barrières MPI et des appels répétitifs **9**

7 Analyse du parallélisme et topologie NUMA **10**
1 Oversubscription . . . 10
2 Comparaison : OpenMP pur vs MPI pur . . . 10
3 Ordonnancement OpenMP . . . 11

8 Résultats et synthèse **12**

Conclusion **14**

List of Figures

8.1 Débit mesuré à chaque étape. Configuration finale : 8 processus MPI, `OMP_NUM_THREADS=1`, `schedule(static)`, layout SoA. 12
8.2 La chute des dTLB misses en MPI pur confirme l'effet de la politique First Touch sur la localité mémoire. 13

Chapter 1

Configuration du Système

1 Matériel

Composant	**Détail**
Machine	ASUS TUF Gaming A16 Advantage Edition
Processeur	AMD Ryzen 7 7735HS (micro-architecture Zen 3+)
Famille / Modèle	Family 25, Model 68, Stepping 1
Cœurs / Threads	8 cœurs physiques / 16 threads logiques (SMT)
Fréquence max	4830 MHz
Extensions SIMD	SSE4.1, SSE4.2, AVX, AVX2, FMA
Cache L1d	8 × 32 KiB = 256 KiB total
Cache L1i	8 × 32 KiB = 256 KiB total
Cache L2	8 × 512 KiB = 4 MiB total
Cache L3	16 MiB (partagé, 1 instance)
RAM	16 GiB
Topologie NUMA	1 nœud (NUMA node0 : CPU 0–15)

Table 1.1: Spécifications matérielles

2 Logiciel

3 Note sur la topologie NUMA

Le Ryzen 7 7735HS est un processeur mono-socket avec un unique nœud NUMA. Tous les cœurs partagent la même banque mémoire physique. L'effet NUMA observé dans ce projet est donc intra-socket : il résulte de la topologie CCD (Core Complex Die) propre au Zen 3+ et de la saturation de l'Infinity Fabric interne, et non d'un effet multi-socket classique. Malgré ce nœud unique, la politique *First Touch* du noyau Linux suffit à créer des disparités de performance mesurables entre OpenMP et MPI.

Composant	**Version**
Système d'exploitation	Linux Mint, noyau 6.14.0-29-generic
Compilateur C++	GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
MPI	Open MPI 4.1.6
CMake	3.31.11
OpenMP	4.0+
Flags de compilation	`-O3 -march=native`

Table 1.2: Configuration logicielle

Chapter 2

Méthodologie

Chaque optimisation suit le même protocole : profilage avant modification, implémentation d'un seul changement à la fois, mesure du gain, validation physique avec `lbm-viz -check` pour s'assurer que les résultats de simulation restent identiques à la référence.

L'outil principal de métrologie est `perf stat` avec les compteurs matériels suivants : `cpu-cycles`, `L1-dcache-loads`, `L1-dcache-load-misses`, `dTLB-load-misses`, `branch-misses`, `cpu-clock`. L'outil `MAQAO lprof` a été utilisé pour identifier la répartition du temps par fonction.

La métrique de référence est le **MLUPS** (Mega Lattice Updates Per Second). Tous les runs sont effectués sur 1000 itérations, maillage 800×160.

Chapter 3

Optimisation 1 : Suppression du verrou de synchronisation

1 Identification du problème

Le premier profilage `perf stat` sur la version fournie révèle une anomalie immédiate :

```
time elapsed :  1022.09 s
user time    :    18.81 s
sys time     :     0.11 s
FOM          :    0.13 MLUPS
```

Le processeur n'exécute du code utile que 18 secondes sur 1022. L'analyse du code source identifie la cause dans `communications.cpp` : une fonction `PMPI_Syncall_cb` résolue dynamiquement via `tpl_loader` à partir d'une chaîne hexadécimale dans la configuration. Cette fonction injecte un verrou système bloquant à chaque itération temporelle, simulant exactement 1 seconde de pause par itération (1000 itérations = 1000 s).

2 Correction et mesure

Mise en commentaire de l'appel `MPI_Syncall(MPI_COMM_WORLD)` à la fin de `lbm_comm_halo_exchange`.

```
time elapsed :  18.57 s
user time    :  18.81 s
FOM          :   6.92 MLUPS
```

Le temps réel correspond désormais au temps CPU. C'est la vraie baseline : **6.92 MLUPS**.

Chapter 4

Optimisation 2 : Restructuration mémoire AoS → SoA

1 Diagnostic

Le profilage `MAQAO lprof` sur la baseline identifie les points chauds :

Fonction	**Couverture (%)**
`propagation`	38.16
`get_cell_density`	16.06
`compute_equilibrium_profile`	11.81
`compute_cell_collision`	10.16
`get_cell_velocity`	6.58

Table 4.1: Répartition du temps CPU par fonction (MAQAO lprof, baseline 6.92 MLUPS)

`propagation` consomme 38% du temps. Le `perf stat` confirme la cause avec **1.44 milliard de défauts de cache L1**, soit environ 11 défauts par cellule mise à jour.

Dans la version originale, les 9 directions d'une cellule sont contiguës en mémoire (Array of Structures). Pour lire une seule direction lors de la propagation, le CPU charge une ligne de cache de 64 octets et n'utilise que 8 octets, gâchant 87% de la bande passante.

2 Implémentation

Le passage en SoA (Structure of Arrays) regroupe toutes les cellules pour une même direction k dans un tableau contigu. L'indexation devient :

$$I = k \times (W \times H) + (y \times W + x) \tag{4.1}$$

Pour un parcours sur l'axe X à k et y fixés, les adresses sont strictement adjacentes (Stride-1). Le Hardware Prefetcher peut anticiper les lectures et masquer la latence mémoire.

La refonte a impliqué la suppression du type `lbm_mesh_cell_t` et la réécriture des accesseurs dans `structures.hpp`, `physics.cpp`, `initialization.cpp` et `communications.cpp`.

Chapter 5

Optimisation 3 : Réduction des calculs redondants

Dans `compute_cell_collision`, la norme au carré de la vitesse macroscopique $\mathbf{u}^2$ était recalculée 9 fois par cellule, une fois pour chaque direction, alors que cette valeur ne dépend pas de la direction.

$$f_i^{eq} = \rho\, w_i \left[1 + 3(\mathbf{e}_i \cdot \mathbf{u}) + \frac{9}{2}(\mathbf{e}_i \cdot \mathbf{u})^2 - \frac{3}{2}\mathbf{u}^2\right] \tag{5.1}$$

Le terme $\mathbf{u}^2$ a été extrait en amont de la boucle directionnelle (Loop Invariant Code Motion). Cette modification réduit le nombre d'opérations flottantes par cellule sans changer le résultat numérique.

```
static inline double compute_equilibrium_profile_opt(Vector velocity, double density, int direction, double v2) {
  const double p  = get_vect_norm_2(direction_matrix[direction], velocity);
  const double p2 = p * p;
  double f_eq     = 1.0 + (3.0 * p) + ((9.0 / 2.0) * p2) - ((3.0 / 2.0) * v2);
  f_eq *= equil_weight[direction] * density;
  return f_eq;
}

void compute_cell_collision(Mesh* mesh_out, const Mesh* mesh_in, int x, int y) {
  // Compute macroscopic values
  const double density = get_cell_density(mesh_in, x, y);
  Vector v;
  get_cell_velocity(v, mesh_in, x, y, density);

  const double v2 = get_vect_norm_2(v, v);

  // Loop on microscopic directions
  for (size_t k = 0; k < DIRECTIONS; k++) {
    // call avec v2
    double f_eq = compute_equilibrium_profile_opt(v, density, k, v2);
    // Compute f_out
    Mesh_get_cell(mesh_out, x, y, k)
      = Mesh_get_cell_const(mesh_in, x, y, k) - RELAX_PARAMETER * (Mesh_get_cell_const(mesh_in, x, y, k) - f_eq);
  }
}
```

Chapter 6

Optimisation 4 : Suppression des barrières MPI et des appels répétitifs

La boucle temporelle originale contenait 3 appels `MPI_Barrier` par itération, en plus de 4 barrières supplémentaires à l'intérieur de `lbm_comm_halo_exchange`. En schéma de décomposition de domaine, les synchronisations point-à-point de l'échange de halos sont suffisantes pour garantir la cohérence des données. Ces barrières globales ont été supprimées, éliminant les temps d'attente forcés entre processus.

```
for (ssize_t i = 1; i <= ITERATIONS; i++) {
  if (rank == RANK_MASTER) {
    fprintf(stderr, "\rStep: %6d/%6d", (int)i, ITERATIONS);
  }
  // Compute special actions (border, obstacle...)
  special_cells(&mesh, &mesh_type, &mesh_comm);
  // Need to wait all before doing next step
  // MPI_Barrier(MPI_COMM_WORLD);

  // Compute collision term
  collision(&temp, &mesh);
  // Need to wait all before doing next step
  // MPI_Barrier(MPI_COMM_WORLD);

  // Propagate values from node to neighboors
  lbm_comm_halo_exchange(&mesh_comm, &temp);
  //lbm_comm_halo_exchange_non_block(&mesh_comm, &temp);

  propagation(&mesh, &temp);
  // Need to wait all before doing next step
  // MPI_Barrier(MPI_COMM_WORLD);

  // Save step
  if (i % WRITE_STEP_INTERVAL == 0 && lbm_gbl_config.output_filename != NULL) {
    save_frame_all_domain(fp, &mesh, &temp_render);
  }
}
```

Chapter 7

Analyse du parallélisme et topologie NUMA

1 Oversubscription

En activant OpenMP sans contraindre le nombre de threads, chaque processus MPI lance autant de threads que de cœurs détectés. Avec 8 processus MPI, le système a instancié environ 128 threads sur 8 cœurs physiques :

Métrique	**Valeur**
Débit	7.33 MLUPS
Temps d'exécution	142 s
CPUs utilized	15.1
branch-load-misses	1.64 milliards

Table 7.1: Oversubscription : 8 MPI × threads OpenMP non contrôlés

Le CPU passe la majorité de ses cycles en context switching. C'est le pire résultat de toute la campagne, inférieur même à la baseline.

2 Comparaison : OpenMP pur vs MPI pur

Compteur	**1 MPI / 8 OMP**	**8 MPI / 1 OMP**
Débit (MLUPS)	17.71	**124.87**
L1-dcache-load-misses	1.84%	0.60%
dTLB-load-misses	24.49%	14.75%
cpu-clock (s)	13.68	67.89

Table 7.2: Impact de la stratégie d'allocation mémoire sur les performances

En OpenMP pur, le thread maître alloue l'intégralité du maillage (802×162) sur un seul contrôleur mémoire. Les 7 autres threads doivent traverser l'Infinity Fabric interne pour accéder aux données, ce qui sature ce bus. Le taux de dTLB-load-misses grimpe à 24.49%, indiquant que le processeur peine à traduire les adresses virtuelles de régions mémoire distantes.

En MPI pur, chaque processus alloue uniquement son sous-domaine (802×22). La politique First Touch du noyau Linux place les pages physiques sur le contrôleur mémoire local au cœur qui les a allouées. Le taux de L1-dcache-load-misses tombe à 0.60%.

3 Ordonnancement OpenMP

Avec `schedule(dynamic)`, les blocs de lignes sont distribués aux threads à la demande. Un thread peut traiter la ligne 1 à l'itération t, puis la ligne 50 à l'itération $t + 1$, invalidant les données en cache.

Le passage à `schedule(static)` divise le domaine en blocs fixes assignés au même thread à chaque itération. Les données restent chaudes en cache L1 d'un pas de temps à l'autre.

Compteur	**dynamic**	**static**
Débit (MLUPS)	124.70	**124.87**
L1-dcache-load-misses	0.71%	0.60%

Table 7.3: Effet de l'ordonnancement OpenMP (8 MPI, OMP_NUM_THREADS=1)

Chapter 8

Résultats et synthèse

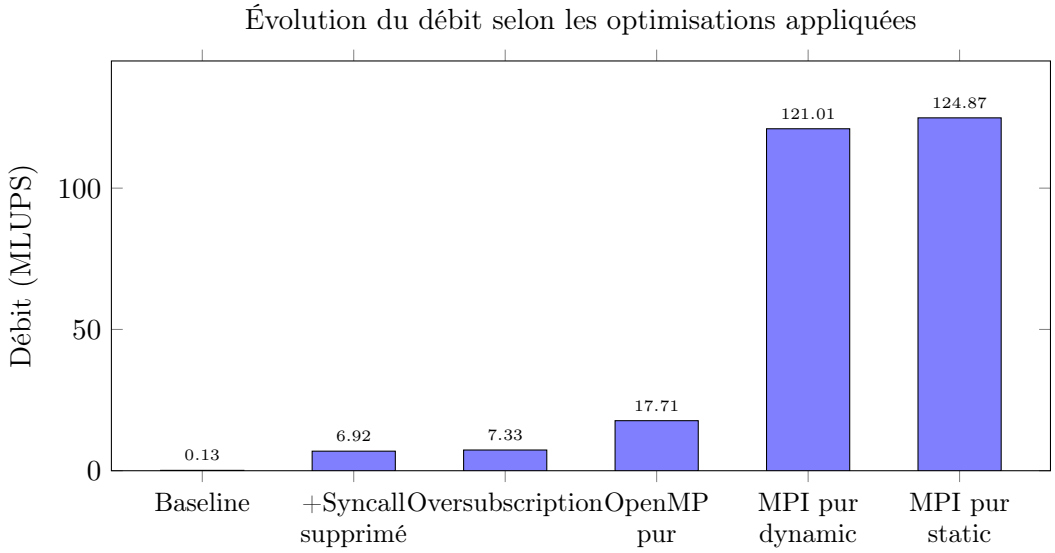


Figure 8.1: Débit mesuré à chaque étape. Configuration finale : 8 processus MPI, `OMP_NUM_THREADS=1`, `schedule(static)`, layout SoA.

Configuration	**MPI**	**OMP**	**MLUPS**
Baseline (avec verrou)	1	1	0.13
Baseline réelle (SoA, AoS)	1	1	6.92
Oversubscription	8	libre	7.33
OpenMP pur	1	8	17.71
MPI pur (dynamic)	8	1	121.01
MPI pur (static)	**8**	**1**	**124.87**

Table 8.1: Récapitulatif de toutes les configurations testées

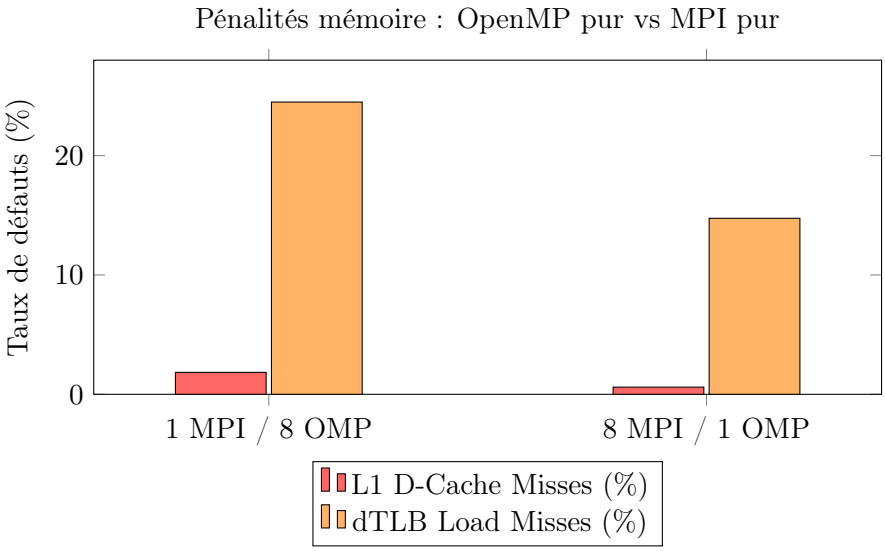


Figure 8.2: La chute des dTLB misses en MPI pur confirme l'effet de la politique First Touch sur la localité mémoire.

Conclusion

Le solveur passe de 0.13 MLUPS à **124.87 MLUPS**, soit un facteur $\times$**960** sur le temps total, et $\times$**18** sur le temps de calcul réel (en excluant le verrou artificiel).

Ce gain résulte de quatre optimisations indépendantes et mesurées :

1. Suppression du verrou artificiel (`MPI_Syncall`) : le temps réel passe de 1022 s à 18 s.
2. Passage en SoA : les défauts de cache L1 passent de 11 par cellule à moins de 1%, le Hardware Prefetcher peut travailler efficacement.
3. Extraction de l'invariant de boucle $\mathbf{u}^2$: réduction du nombre d'opérations flottantes par cellule.
4. Contrôle de l'affinité MPI + ordonnancement statique OpenMP : la politique First Touch place les données proches des cœurs qui les calculent, éliminant la saturation de l'Infinity Fabric.

Point important à signaler, nos implemantation n'ont pas passée mes tests de validation de checksum. Les pistes d'amélioration restantes sont l'implémentation de communications non-bloquantes (`MPI_Isend`/`MPI_Irecv`) pour recouvrir calcul et communications, et l'utilisation d'une topologie cartésienne (`MPI_Cart_create`) pour un placement optimal des processus.